Task 1: Guided Activity, A generic Stack

```
import Foundation
struct Stack<Element> {
    private var items: [Element] = []
    mutating func push(_ item: Element) {
        items.append(item)
    }
   mutating func pop() -> Element? {
        return items.popLast()
    }
    var top: Element? {
        return items.last
    }
    var isEmpty: Bool {
        return items.isEmpty
    }
    var count: Int {
        return items.count
    }
}

var intStack = Stack<Int>()
intStack.push(10)
intStack.push(20)
intStack.push(30)

print("Integer Stack Count:", intStack.count)
print("Integer Stack Top:", intStack.top!)
print("Popped Integer:", intStack.pop()!)
print("Integer Stack Is Empty:", intStack.isEmpty)

var stringStack = Stack<String>()
stringStack.push("Swift")
stringStack.push("UIKit")

print("String Stack Count:", stringStack.count)
print("String Stack Top:", stringStack.top!)
print("Popped String:", stringStack.pop()!)

func largest<T: Comparable>(in items: [T]) -> T? {
    return items.max()
}
```

```
print("Largest Integer:", largest(in: [10, 50, 30])!)
print("Largest String:", largest(in: ["Apple", "Mango", "Banana"])!)

func swapValues<T>(_ a: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}

var first = 100
var second = 200

swapValues(&first, &second)
print("After Swap:", first, second)

var word1 = "Hello"
var word2 = "Swift"

swapValues(&word1, &word2)
print("After String Swap:", word1, word2)

struct Student {
    var name: String
}
```

OUTPUT:

```
Integer Stack Count: 3
Integer Stack Top: 30
Popped Integer: 30
Integer Stack Is Empty: false
String Stack Count: 2
String Stack Top: UIKit
Popped String: UIKit
Largest Integer: 50
Largest String: Mango
After Swap: 200 100
After String Swap: Swift Hello
```

Task 2: Practice Task, ATM withdrawal with errors

```
import Foundation
enum ATMError: Error {
    case insufficientFunds(shortBy: Double)
    case invalidAmount
    case dailyLimitExceeded
}
struct Account {
    var balance: Double
```

```
    var withdrawnToday: Double = 0
    let dailyLimit: Double = 20000
    mutating func withdraw(_ amount: Double) throws {
        if amount <= 0 {
            throw ATMError.invalidAmount
        }
        if withdrawnToday + amount > dailyLimit {
            throw ATMError.dailyLimitExceeded
        }
        if amount > balance {
            throw ATMError.insufficientFunds(
                shortBy: amount - balance
            )
        }
        balance -= amount
        withdrawnToday += amount
        print("Withdrawal successful: ₹\(amount)")
        print("Remaining balance: ₹\(balance)")
    }
}

var account = Account(balance: 10000)
let amounts: [Double] = [500, -10, 25000, 15000]
for amount in amounts {
    print("\nTrying to withdraw: ₹\(amount)")
    do {
        try account.withdraw(amount)
    } catch ATMError.insufficientFunds(let shortBy) {
        print("Error: Insufficient funds. Short by ₹\(shortBy)")
    } catch ATMError.invalidAmount {
        print("Error: Amount must be greater than zero.")
    } catch ATMError.dailyLimitExceeded {
        print("Error: Daily withdrawal limit of ₹20000 exceeded.")
    } catch {
        print("An unexpected error occurred.")
    }
}

print("\nTesting try?")
if (try? account.withdraw(100)) != nil {
    print("Withdrawal worked.")
} else {
    print("Withdrawal failed.")
}
```

```
print("\nFinal balance: ₹\(account.balance)")
```

OUTPUT:

```
Trying to withdraw: ₹500.0
Withdrawal successful: ₹500.0
Remaining balance: ₹9500.0

Trying to withdraw: ₹-10.0
Error: Amount must be greater than zero.

Trying to withdraw: ₹25000.0
Error: Daily withdrawal limit of ₹20000 exceeded.

Trying to withdraw: ₹15000.0
Error: Insufficient funds. Short by ₹5500.0

Testing try?
Withdrawal successful: ₹100.0
Remaining balance: ₹9400.0
Withdrawal worked.

Final balance: ₹9400.0
```

Task 3: Challenge Task, Result type and defer

```
import Foundation
enum AgeError: Error {
    case empty
    case notANumber
    case outOfRange
}
func parseAge(_ text: String) throws -> Int {
    guard !text.isEmpty else {
        throw AgeError.empty
    }
    guard let age = Int(text) else {
        throw AgeError.notANumber
    }
    guard age >= 1 && age <= 120 else {
        throw AgeError.outOfRange
    }
    return age
}

let result = Result {
    try parseAge("25")
}
switch result {
case .success(let age):
```

```
    print("Valid age:", age)
case .failure(let error):
    switch error {
    case AgeError.empty:
        print("Error: Age cannot be empty.")
    case AgeError.notANumber:
        print("Error: Input must be a number.")
    case AgeError.outOfRange:
        print("Error: Age must be between 1 and 120.")
    default:
        print("Unknown error:", error)
    }
}
enum ReportError: Error {
    case failed
}
func saveReport() throws {
    print("Opening file")
    defer {
        print("Closing file")
    }
    print("Saving report")
    throw ReportError.failed
}
do {
    try saveReport()
} catch {
    print("Error: Failed to save report.")
}
```

OUTPUT:

```
Valid age: 25
Opening file
Saving report
Closing file
Error: Failed to save report.
```